Blobli — Guía de estudio

Implementación sonora de una región de juego · TP1

Materia: Sonido y Música para Videojuegos · UADE
Proyecto: `Blobli.wproj` · Wwise 2023.1.19
Integrantes: Juani, Nacho, Zeki, Mati
El proyecto está dividido en cuatro áreas de responsabilidad. Cada una se lleva un componente de la consigna, un mecanismo técnico que no usa ninguna otra, y una porción pareja de objetos.
Cada parte explica qué hay en tu área, cómo funciona, por qué se decidió así y qué te pueden preguntar. La Parte 0 la tienen que saber los cuatro.

> **Cómo usar esta guía**
> **1.** Los cuatro leen la **Parte 0**: son los cinco conceptos sobre los que está armado todo. Sin eso no se puede defender ninguna parte.
> **2.** Cada uno estudia **su área** hasta poder explicarla sin leer, y la abre en Wwise mientras la lee.
> **3.** Cada uno le cuenta su área a los otros tres. Si no se entiende contada, no se entiende.
> **4.** Repasen juntos el **recorrido del Soundcaster** del final: es literalmente lo que el profesor va a apretar.
> **5.** Las preguntas de los recuadros "Te pueden preguntar" son las que yo haría si evaluara esto. Si las cuatro personas responden las suyas, no hay por dónde entrarles.

Parte 0 · Lo que tienen que saber los cuatro

Blobli es un puzzle 2D por turnos sobre una grilla. El personaje se mueve de celda en celda, la luz lo mata, junta gemas que proyectan sombras y tiene que llegar al Altar. El turno sólo avanza cuando el jugador se mueve: **el mundo no corre solo**. Esa sola característica explica la mitad de las decisiones del proyecto.

Cómo se dividió el trabajo

Cuatro áreas. Cada una tiene un componente de la consigna, un mecanismo técnico propio y una parte de los Events.

Área	Quién	Req	Mecanismo propio	Obj.	Ev.
1. Personaje, Events y Soundcaster	Juani	**1**	Switch Container en *Step* + Random en *Shuffle*	14	13
2. Ambiente, clima y fauna	Nacho	**2**	Blend Track con crossfade + *Use Game Parameter*	15	2
3. Acciones, interfaz y variable de gameplay	Zeki	**4**	Curvas RTPC sobre un Game Parameter	13	6
4. Recinto, mezcla y espacio	Mati	**3**	States + Buses + Auxiliary Bus + reverb	6 + 6 buses	2

Los Events y los objetos se superponen un poco entre áreas, porque el proyecto no está hecho de compartimentos. Al final de la guía hay una tabla de **zonas compartidas** con quién explica qué en cada caso.

Los cinco conceptos

1. Las dos jerarquías

Wwise tiene dos árboles separados y es lo primero que hay que tener claro. La **Actor-Mixer Hierarchy** es el *contenido*: qué sonidos existen y cómo se eligen entre sí. La **Master-Mixer Hierarchy** es la *mezcla*: por qué canales pasan y cómo suenan en conjunto. Un objeto de la primera siempre sale por un bus de la segunda.

2. Event

Es la única puerta por la que el juego le habla a Wwise. Unity nunca dice "reproducí este archivo": dice `PostEvent("Play_Footstep")` y Wwise decide el resto. Un Event puede tener **varias acciones**: `Set_Recinto_Altar` hace un Set State y un Set Switch a la vez.

3. Los tres tipos de Game Sync

Tipo	Qué representa	En Blobli
Switch	una decisión discreta: es A o es B, no hay punto medio	`Tile` = Ground / Mushroom / Shadow
State	un estado global que cambia *cómo suena todo*, no qué suena	`Recinto` = Exterior / Altar
RTPC	un número continuo que modula propiedades en tiempo real	`SunExposure` y `Turns`, de 0 a 100

La diferencia Switch / State es la que más se confunde: **el Switch cambia QUÉ se oye, el State cambia CÓMO suena el espacio.** Se puede estar en la zona del Altar oyéndola desde afuera: por eso son dos cosas distintas.

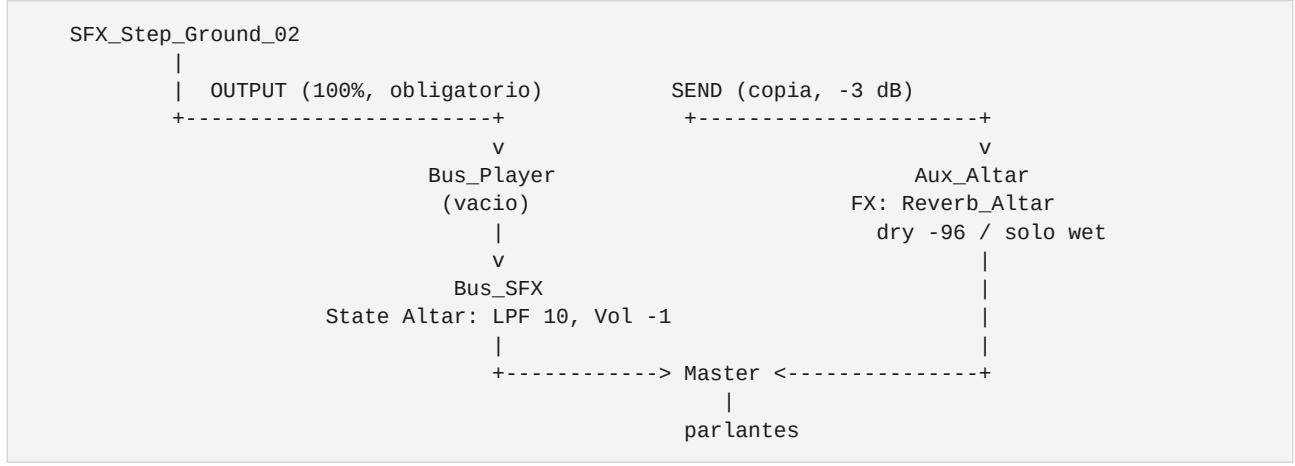
4. Bus y Aux Bus

Un **Aux Bus es un bus**: tiene las mismas capacidades. Lo que cambia es **la puerta por la que entra el sonido**, y hay exactamente dos:

	Output	Send
Qué manda	el **100%** de la señal	una **copia**, al nivel que elijas

	Output	Send
Cuántos puede tener un sonido	**exactamente uno**, obligatorio	**cero o muchos**, opcional
Qué significa	*dónde vive* ese sonido	*a dónde además mando una copia*
A qué tipo de bus llega	a un Audio Bus normal	**sólo** a un Auxiliary Bus

Esa última fila es la regla dura: un Audio Bus normal **no puede recibir sends**. Por eso existe el tipo Auxiliary.



El sonido **se parte en dos** y las dos mitades se vuelven a juntar en el Master: la izquierda es la pisada seca, la derecha es sólo su reverb. Por eso el `DryLevel` del efecto está en -96 dB: lo seco ya llega por el otro camino.

5. Contenedores

Contenedor	Qué hace	Dónde se usa
Random	elige UNO de sus hijos cada vez. En *Shuffle* recorre todos antes de repetir	pisadas, gemas, fauna
Switch	elige una rama según un Switch Group	`SW_Steps_Tile`, `SW_Zone_Ambience`, `SW_Fauna`
Blend	suena TODOS sus hijos a la vez. Con un Blend Track los cruza según un RTPC	`BLD_Field`, `BLD_Altar`, `BLD_Fauna`

> **La frase que más rinde**
> Hay una regla que se repite en todo el proyecto y conviene decirla con estas palabras: **el Switch decide qué existe, el State decide cómo suena, el RTPC decide cuánto.**
> Si les preguntan "¿por qué acá usaron esto y no un RTPC?", la respuesta sale de ahí.

El mapa completo

```
ACTOR-MIXER HIERARCHY

AMX_Player ....... Bus_Player   [1]
 +- SW_Steps_Tile          Switch: Tile
 |   +- RND_Step_Ground      Shuffle, 3
 |   +- RND_Step_Mushroom    Shuffle, 3
 |   +- RND_Step_Shadow      Shuffle, 3
 +- [Actions]              [3]
     +- RND_Gem_Pickup       Shuffle, 4
     +- SFX_Shadow_Place
     +- SFX_Key_Place
     +- SFX_Death

AMX_Ambience ..... Bus_AMB
 +- SW_Zone_Ambience   [4]  Switch: Zone
     +- BLD_Field      [2]  Blend Track "Sun"
     |   +- SFX_Wind_Soft     0-40
     |   +- SFX_Wind_Med      25-75
     |   +- BLD_Sun_High      60-100
     |   |   +- SFX_Wind_Hot
     |   |   +- SFX_Sun_Sizzle
     |   +- SW_Fauna   [2]  Switch: Fauna
     |       +- BLD_Fauna
     |           +- RND_Insects    2 + silencio
     |           +- RND_Mushrooms  2 + silencio
     +- BLD_Altar      [4]
     |   +- SFX_Altar_Aura / Static / Voices
     +- SFX_Void       [4]  en las DOS zonas

AMX_UI ........... Bus_UI       [3]
 +- SFX_Turn_Pulse / UI_Hover / UI_Click / Win_Level
```

```
MASTER-MIXER HIERARCHY

Master Audio Bus          [4]
 +- Bus_SFX    <- State Recinto
 |   +- Bus_Player
 |   +- Bus_AMB
 |
 +- Bus_UI     (sin State, sin send)
 |
 +- Aux_Altar  <- Reverb_Altar
                  el State decide
                  si el retorno se oye

GAME SYNCS

 Game Parameters
   SunExposure  0-100   [2]
   Turns        0-100   [3]

 Switch Groups
   Tile   G/M/S         [1]
   Zone   Field/Altar   [4]
   Fauna  Active/Silent [2]

 State Group
   Recinto None/Ext/Altar [4]

  Events: 13            [1]

[n] = area responsable
```

Área 1 · Juani — Personaje, Events y Soundcaster

Consigna 1: footsteps en distintas superficies, evitando la reiteración. Además: la estructura completa de Events y la sesión de prueba.

Qué hay en tu área

Objeto	Qué es	Configuración
`SW_Steps_Tile`	Switch Container sobre `Tile`	Play Mode **Step** · Default `Ground` · Volume -2.5 dB
`RND_Step_Ground`	Random, piso duro	**Shuffle** · randomizer de Volume +/-1 dB · 3 archivos de ~57 ms
`RND_Step_Mushroom`	Random, hongo	igual · 3 archivos de ~290 ms
`RND_Step_Shadow`	Random, sombra	igual · 3 archivos de ~235 ms
`AMX_Player`	raíz del personaje	sale por `Bus_Player` · Send a `Aux_Altar` a -3 dB
Los 13 Events	toda la Events Work Unit	17 acciones en total
`SC_Blobli_TP1`	sesión de Soundcaster	13 módulos, 2 sliders, 2 selectores

Cómo funcionan las pisadas

El juego hace **Set Switch** `Tile` al entrar a la celda y recién después dispara el Event. El Switch Container ya apunta a la rama correcta, así que el cambio de superficie se oye en la pisada siguiente.

```
Unity:  SetSwitch("Tile","Mushroom")  ->  PostEvent("Play_Footstep")

        SW_Steps_Tile  (Step: un Play = una pisada)
              |
              +--> RND_Step_Mushroom  (Shuffle)
                            |
                            +--> una de las 3, nunca la misma dos veces seguidas
                                 + randomizer de Volume de +/-1 dB
```

- **Play Mode Step y no Continuous.** En Step cada Play reproduce una pisada y avanza. En Continuous un solo Play caminaría para siempre. El ritmo lo marca el juego (un paso por celda), no el contenedor.
- **Shuffle y no Random Standard.** Shuffle recorre las tres variaciones antes de repetir ninguna. Con Standard podés oír la misma pisada tres veces seguidas: el "efecto ametralladora" que la consigna pide evitar.
- **El randomizer de Volume va en el CONTENEDOR.** Así se hereda a los tres hijos y hay un solo lugar que mirar. Un randomizer en el contenedor y otro en el hijo sobre la misma propiedad *se suman*.
- **Es de sólo +/-1 dB** porque los nueve archivos ya están emparejados en sonoridad. Está para romper la repetición mecánica, no para corregir niveles.
- **No hay randomizer de Pitch.** Se probó y se sacó: en una pisada corta el cambio de tono se percibe como un error, no como variedad.

El tratamiento de los archivos

Superficie	Pasa-altos	Recorte	Fade-out	Duración final
Ground	60 Hz, **1er orden**	-45 dB	6 ms	~57 ms
Mushroom	150 Hz, 2do orden	-24 dB	8 ms	~290 ms
Shadow	150 Hz, 2do orden	-24 dB	8 ms	~235 ms

Los nueve quedaron en **-25.75 LUFS**. Se usó LUFS y no RMS porque el RMS trata igual a un click de 57 ms y a un sonido de 290 ms, y el oído no: integra sobre unos 200 ms. Medidos por RMS los tres grupos daban idénticos y Ground se oía **8 dB más bajo**.

Los Events

Trece Events. Lo que conviene saber explicar es que **un Event puede tener varias acciones ordenadas**:

```
Play_Death    Play SFX_Death  +  Stop SW_Zone_Ambience   (Delay 0.2 s, Fade 1.5 s)
Play_Win      Play SFX_Win    +  Stop SW_Zone_Ambience   (Delay 0.5 s, Fade 2.5 s)

Set_Recinto_Altar      Set State  Recinto = Altar
                       Set Switch Zone    = Altar
```

El Delay hace que el ambiente empiece a retirarse *después* de que arrancó el sonido de muerte, no al mismo tiempo: una pequeña puesta en escena resuelta enteramente desde el Event, sin tocar un solo sonido.

> **Te pueden preguntar**
>
> **¿Por qué Ground dura 57 ms y las otras 290?** Porque un piso duro *es* un golpe corto y un hongo es blando: la diferencia de duración es la información que le dice al jugador qué está pisando. Se evaluó igualarlas con un Time Stretch y se descartó: sería un estiramiento de casi 10x, muy por encima de lo que el algoritmo tolera.
>
> **¿Por qué Ground lleva un filtro de primer orden?** Porque el de segundo orden con Q = 0.707 es subamortiguado: sobrepasa y oscila cerca del corte, y en un sonido tan corto esa oscilación queda al descubierto como una vibración al final. El de primer orden tiene un solo polo real y no puede oscilar.
>
> **¿Con tres variaciones alcanza?** Con Shuffle sí: garantiza que no se repita ninguna hasta agotar las tres. Con dos la secuencia sería siempre A-B-A-B, que vuelve a ser predecible.

Área 2 · Nacho — Ambiente, clima y fauna

Consigna 2: recrear un bioma, incluir elementos del paisaje sonoro, incorporar un sistema dinámico de clima y randomizar algunos sonidos para evitar repetición.

Qué hay en tu área

Objeto	Qué es
`BLD_Field` + Blend Track "Sun"	el sistema de clima: cruza tres capas según `SunExposure`
`SFX_Wind_Soft` / `Med` / `Hot`	las tres capas de viento
`BLD_Sun_High` + `SFX_Sun_Sizzle`	la capa de sol alto: viento caliente + chisporroteo, agrupados
`SW_Fauna`	Switch Container con *Use Game Parameter* sobre `SunExposure`
`RND_Insects` / `RND_Mushrooms`	los dos sistemas intermitentes del claro
Los dos `SIL_*_Gap`	fuentes *Wwise Silence* que abren los huecos largos

El clima: un Blend Track con crossfade

Las tres capas no se encienden y apagan: se **cruzan**. El Blend Track tiene *Enable Crossfading* y usa `SunExposure` como entrada. Los tramos se superponen, así que nunca hay un punto donde no suene nada.

```
SunExposure   0 ------ 25 --- 40 ------ 60 --- 75 ------ 100

Wind_Soft    [=================]
Wind_Med               [===================]
BLD_Sun_High                        [=================]
                                     (Wind_Hot + Sun_Sizzle)

SW_Fauna     [ suena siempre, NO esta en el blend track ]
```

Un solo número mueve el bioma entero: sube el sol, el viento se vuelve caliente **y** el claro se queda sin bichos.

La fauna: el truco del switch vacío

`SW_Fauna` tiene dos switches, pero `Silent` **no tiene nada asignado**. Cuando `SunExposure` pasa de 65, la fauna directamente no se instancia.

	Switch con el tramo vacío	RTPC de volumen a -200 dB
¿Se oye?	No	No
¿Existe la instancia?	**No**	**Sí**
¿Gasta una voice?	No	Sí
¿Decodifica audio?	No	Sí

Las dos opciones suenan igual, pero con el RTPC los dos Random Containers siguen vivos gastando CPU para producir silencio. **Esa es la justificación que hay que saber decir.** El mismo criterio gobierna `SW_Zone_Ambience`: en el claro no existen las capas del Altar.

Los intermitentes

	`RND_Insects`	`RND_Mushrooms`
Play Mode	Continuous + Loop Infinite	Continuous + Loop Infinite
Transition	Delay 7 s, randomizer -3/+5 -> **4 a 12 s**	Delay 11 s, randomizer -5/+7 -> **6 a 18 s**
Hijos	2 one-shots (peso 30) + Silence (peso **40**)	2 one-shots + Silence, pesos por defecto
Limit repetition	activo, 1	activo, 1

	`RND_Insects`	`RND_Mushrooms`
Randomizers	Pitch +/-200 · Volume +/-3 · LPF 0 a +30	Pitch +/-250 · Volume +/-2.5 · LPF 0 a +25

- **El Loop va en el CONTENEDOR, no en los hijos.** Un hijo que loopea no termina nunca y el contenedor jamás dispararía el siguiente.
- **Delay y no Trigger Rate.** El Delay cuenta desde que *termina* un elemento, así que dos bichos nunca se pisan. El Trigger Rate es de inicio a inicio y produce cadencias regulares, que es lo que no queremos.
- **El Delay va randomizado**: fijo sonaría a metrónomo.
- **Dos sistemas con cadencias distintas** (4-12 s y 6-18 s) para que los patrones no se alineen y el claro no suene a bucle.
- **El randomizer de Low-pass tiene lectura física**, no es adorno: cada bicho suena a una distancia distinta, y lo primero que se pierde con la distancia son los agudos.
- **Los randomizers viven en un solo nivel.** Estaban también en cada hijo y los dos se suman: el pitch de los hongos llegó a oscilar once semitonos.

> **Te pueden preguntar**
>
> **¿Por qué la fauna usa *Use Game Parameter* y la zona no?** Porque el juego no tiene ningún evento que avise "subió el sol": hay que deducirlo de la magnitud. En cambio "el jugador entró al Altar" sí es un evento, y conviene que lo setee el Event.
> **¿Qué es el *Render* de los efectos del sol?** `SFX_Sun_Sizzle` tiene un Pitch Shifter y un Time Stretch, los dos plugins más caros de Wwise, sobre una cama que loopea sin parar. Como ningún RTPC los modula se puede tildar *Render*: Wwise hornea el efecto en la conversión, suena igual y cuesta cero CPU en runtime.
> **¿Por qué** `SW_Fauna` **no está en el Blend Track?** Porque la fauna no es una capa de clima que haya que cruzar: está o no está. Esa decisión la toma su propio Switch Group.

Área 3 · Zeki — Acciones, interfaz y variable de gameplay

Consigna 4: implementar la interacción de al menos un sonido con una variable del juego. Además: todo lo que hace que el proyecto suene a juego y no a maqueta.

Qué hay en tu área

Objeto	Qué es	Volume
`RND_Gem_Pickup` (4)	recoger una gema. Shuffle, *Limit repetition* 2, Pitch +/-200, Volume +/-1.5	-4 a -9 dB
`SFX_Shadow_Place`	proyectar una sombra: la mecánica central. Sin randomizar a propósito	-10 dB
`SFX_Key_Place`	colocar una Key gem	-13 dB
`SFX_Death`	Blobli queda expuesto a la luz	0 dB
`SFX_Turn_Pulse`	el pulso de turno. Lleva 2 de las 7 curvas RTPC	-1.5 dB
`AMX_UI` + 3 sonidos	hover, click y jingle de victoria	-7 / -4.5 / -1 dB
Las 7 curvas RTPC	sobre el Game Parameter `Turns`	—

Turns: la variable de gameplay

`Turns` son los turnos transcurridos, normalizados de 0 a 100. La elección de *turnos* y no de *segundos* es el argumento central: Blobli es por turnos, así que el mundo sólo avanza cuando el jugador se mueve. Atado al reloj, el juego apuraría al jugador mientras piensa; atado a los turnos, **la tensión es consecuencia directa de sus decisiones**.

Objeto	Propiedad	Turno 0	Turno 100	Qué se oye
`SFX_Void`	Volume	-10 dB	-4 dB	el vacío sube 6 dB
`SFX_Void`	Low-pass	45	0	el vacío se abre y gana agudos
`BLD_Fauna`	Volume	0 dB	-10 dB	la fauna se apaga
`RND_Insects`	Delay	4-12 s	12-20 s	los bichos se espacian
`RND_Mushrooms`	Delay	6-18 s	16-28 s	los hongos se espacian
`SFX_Turn_Pulse`	Volume	-9 dB	0 dB	el pulso aparece
`SFX_Turn_Pulse`	Pitch	0	+500 cents	el pulso sube 5 semitonos

> **La curva es un desplazamiento, no un valor**
>
> En propiedades aditivas (Volume, Pitch, Low-pass, Delay) la curva **no** dice "poné esto": dice "sumale esto al valor base". `RND_Insects` tiene Delay base 7 s; la curva en el turno 100 vale +8; el resultado son 15 s, y encima sigue el randomizer de -3/+5.
>
> Dato técnico: las dos curvas de Delay sólo funcionan porque los contenedores tienen *Play Mechanism* > *Special Transitions* **activado** y en tipo *Delay*. Apagado, la curva existiría y no haría nada, sin ningún error que avisara.

La idea: la tensión se construye sacando

```
turno 0                          turno 100
-------                          ---------
bichos cada 4-12 s               bichos cada 12-20 s   (se van)
hongos cada 6-18 s               hongos cada 16-28 s   (se van)
fauna a 0 dB                     fauna a -10 dB        (se apaga)
void a -10 dB, opaco             void a -4 dB, abierto (aparece)
pulso inaudible                  pulso presente y agudo
```

El claro está vivo al principio. A medida que se gastan turnos esa vida se retira y lo que queda abajo —el void— sube y se destapa. **No entra música dramática: se va el ruido de fondo que te hacía compañía.**

La nivelación de los one-shots

Los archivos de acciones e interfaz llegaron con **17 dB de diferencia** entre el más bajo y el más alto. Se midió el **LUFS momentáneo máximo** de cada uno (ventana de 400 ms, que es la métrica que corresponde a un sonido corto) y se compensó con *Voice Volume* en cada objeto, **sin tocar los archivos**. Los cuatro `GEM_Pickup` quedaron todos al mismo nivel: dentro de un Random Container, variaciones de distinto volumen se perciben como un error y no como variedad. **Sólo se atenúa, nunca se levanta**, para que ninguna voice pueda clipear.

> **Te pueden preguntar**
>
> **¿Por qué las gemas se randomizan y** `SFX_Shadow_Place` **no?** Porque recoger una gema pasa muchas veces y hay que evitar la repetición; proyectar una sombra es la jugada central y el jugador tiene que reconocerla siempre idéntica como confirmación de que salió bien.
> **¿Por qué siete curvas si la consigna pide una?** Porque con una sola el efecto no se percibe como un sistema. Con `SFX_Void` solo ya estaría cumplido; las otras seis son las que convierten un requisito en una progresión dramática.
> **¿Por qué el pulso sube de tono y no sólo de volumen?** Porque el Pitch en Wwise es resampling: además de subir el tono acorta la duración. El pulso se vuelve más agudo y más breve a la vez, que es exactamente la sensación de apuro.

Área 4 · Mati — Recinto, mezcla y espacio

Consigna 3: implementar con States y un efecto de reverb la emulación de un recinto distinto. Es el área con menos objetos y más arquitectura: toda la Master-Mixer es tuya.

Qué hay en tu área

Objeto	Qué es
`BLD_Altar` (3 capas)	Blend agrupador: `Aura`, `Static` y `Voices` suenan siempre juntas
`SFX_Void`	la cama grave del mundo. Asignada a las **dos** zonas, con *Continue to play*
`SW_Zone_Ambience`	Switch Container en **Continuous** que intercambia las dos zonas
State Group `Recinto`	`None` / `Exterior` / `Altar` · Default Transition Time 0.8 s
Los 5 buses + el Aux	`Bus_SFX`, `Bus_Player`, `Bus_AMB`, `Bus_UI`, `Master`, `Aux_Altar`
`Reverb_Altar`	ShareSet de Wwise RoomVerb

Cómo funciona el recinto

Al cruzar el umbral de la cámara el juego dispara **un solo Event** con dos acciones:

```
Set_Recinto_Altar      Set State  Recinto = Altar     (COMO suena el espacio)
                       Set Switch Zone    = Altar     (QUE se oye)

Set_Recinto_Exterior   Set State  Recinto = Exterior
                       Set Switch Zone    = Field
```

El **State** no crea ni destruye nada: aplica desplazamientos globales sobre objetos que ya están sonando.

Qué toca el State	Exterior / None	Altar	Efecto
`Aux_Altar` · Volume	-96 dB	0 dB	el retorno de reverb aparece
`Bus_SFX` · Low-pass	0	10	el mundo se apaga un poco
`Bus_SFX` · Volume	0 dB	-1 dB	deja aire para la cola de reverb

Y el **Switch** cambia la cama: `BLD_Field` sale, `BLD_Altar` entra, con fades de 2 s. `SFX_Void` está asignado a las dos zonas con *Continue to play*, así que **no se reinicia** al cruzar: es el hilo que cose las dos zonas y hace que la transición se perciba como un cambio de lugar y no como un corte.

Los buses: la respuesta que sorprende

Mirando los cinco buses, **ninguno tiene una sola propiedad configurada**. Están todos en default, ni siquiera un fader movido. Sólo dos tienen algo encima, y en los dos casos a través del State:

```
Master Audio Bus    propiedades: NINGUNA   (solo el Audio Device)
+- Bus_SFX          propiedades: NINGUNA   <- pero tiene el State Recinto
|  +- Bus_Player    propiedades: NINGUNA
|  +- Bus_AMB       propiedades: NINGUNA
+- Bus_UI           propiedades: NINGUNA
+- Aux_Altar        propiedades: NINGUNA   <- pero tiene el efecto + el State
```

Eso no es un descuido: es el punto. Un bus no necesita tener nada configurado para servir. Su trabajo es *existir*, para que las cosas se agrupen debajo y haya un lugar donde actuar. `Bus_Player` y `Bus_AMB` están vacíos y son los dos más útiles del proyecto: son el punto de agarre, y hacen que `Bus_SFX` signifique "todo lo que ocurre dentro del mundo", que es justo el conjunto que el State tiene que oscurecer.

> **La decisión de ruteo más importante del proyecto**
>
> `Bus_UI` cuelga directamente del Master y **no** de `Bus_SFX`. No lleva State ni send a la reverb.
> Lo no diegético —hover, click, jingle de victoria, pulso de turno— no ocurre dentro del mundo: es información para el jugador. Si colgara de `Bus_SFX` se oscurecería y se llenaría de reverb al entrar al Altar, y el jugador perdería claridad en la interfaz justo en el momento más importante del nivel.
> **No hay ningún parámetro que haga esto: lo hace el lugar donde está colgado.** Eso es arquitectura.

El reverb en números

Parámetro	Valor	Por qué
`DecayTime`	2.2 s	cámara de piedra mediana, no catedral
`RoomSize`	55	idem
`PreDelay`	18 ms	separa el directo de la primera reflexión
`ERLevel`	-6 dB	las primeras reflexiones son lo que el oído lee como "sala" en un sonido corto
`ReverbLevel`	-3 dB	nivel de la cola
`DryLevel`	-96 dB	vive en un Aux: sólo debe devolver señal mojada
`HFDamping`	3	la piedra refleja agudos; con damping alto el recinto no se distinguía

Los sends son distintos por fuente: `AMX_Player` manda **-3 dB** y `AMX_Ambience` **-10 dB**. Las pisadas se mojan más que el ambiente porque son las que tienen que delatar el espacio.

> **Te pueden preguntar**
>
> **¿Por qué el reverb en un Aux y no como insert en cada sonido?** Tres razones: una sola instancia del efecto procesa a todos los emisores; podés dosificar cuánto se moja cada fuente; y el State sube o baja el *retorno* sin crear ni destruir nada. Como insert habría una instancia de reverb por cada sonido sonando, el 100% de la señal pasaría por el efecto y no habría señal seca.
> **¿Por qué un State y no bajar el volumen a mano?** Porque un State es un *mixer snapshot*: cambia propiedades de lo que ya está sonando, globalmente, sin tocar cada objeto. Y si hubiera varios States activos, sus desplazamientos se suman.
> **¿Qué pasa con el State `None`?** Lo crea Wwise solo. Se le dio el mismo valor que a `Exterior` (-96 dB) para que, si el juego nunca setea el State, el proyecto igual suene bien.

Zonas compartidas y cierre

Objetos que toca más de un área

El proyecto no está hecho de compartimentos. Conviene tenerlo claro porque el profesor puede preguntarle a cualquiera por cualquier cosa:

Objeto	Lo comparten	Quién explica qué
`SFX_Void`	Áreas 4 y 3	Mati: por qué suena en las dos zonas sin reiniciarse. Zeki: sus dos curvas de `Turns`
La fauna	Áreas 2 y 3	Nacho: el switch vacío y los intermitentes. Zeki: cómo `Turns` los espacia
`Bus_UI`	Áreas 4 y 3	Mati: por qué está fuera de `Bus_SFX`. Zeki: qué cuelga de él
Los Events	Área 1 y todas	Juani: la estructura y las acciones múltiples. Cada uno: qué dispara lo suyo
`SW_Zone_Ambience`	Áreas 4 y 2	Mati: el intercambio de zonas. Nacho: qué hay dentro de `BLD_Field`

El recorrido del Soundcaster

Sesión `SC_Blobli_TP1`: los 13 Events como módulos, sliders de `SunExposure` y `Turns`, selector de Switch `Tile` y selector de State `Recinto`. Arranca en `Exterior`, zona `Field`, baldosa `Ground`. Practíquenlo hasta que salga solo.

#	Paso	Qué demuestra	Área
1	`Play_Ambience`	el claro con el sol bajo: void, viento quieto y fauna	2
2	Barrer `SunExposure` de 0 a 100	el **clima dinámico**. Al pasar 65 el selector `Fauna` cambia solo y los bichos se van	2
3	Volver a 20	la fauna regresa con fade de 3 s	2
4	`Set_Recinto_Altar`	el **recinto**: la cama cruza y entra la reverb con una sola acción. El void no se reinicia	4
5	`Play_Footstep` en el Altar	el mismo archivo vuelve con cola: sólo cambió el retorno del Aux	4
6	`Play_UI_Click` en el Altar	suena limpio: `Bus_UI` está fuera de `Bus_SFX`	4
7	`Set_Recinto_Exterior`	el regreso, también de una sola acción	4
8	`Play_Footstep` cambiando `Tile`	los **footsteps**: tres timbres y variación dentro de cada uno	1
9	Subir `Turns` de 0 a 100	la **variable de gameplay**: el void sube y se abre, la fauna baja y se espacia	3
10	`Play_Death`	un Event de dos acciones ordenadas con Delay	1

Glosario de bolsillo

Término	En una línea
RTPC	Real-Time Parameter Control: una curva que conecta un número del juego con una propiedad de audio
Game Parameter	el número en sí. En Blobli: `SunExposure` y `Turns`
Voice	una instancia de un sonido reproduciéndose. Cada voice cuesta CPU, aunque esté inaudible
Output	el bus donde vive un sonido. Uno solo, obligatorio, pasa el 100%
Send	una copia de la señal a un Auxiliary Bus. Cero o muchos, opcional
ShareSet	un efecto configurado una vez y reutilizado por varios objetos
Mixer snapshot	otra forma de decir State: desplazamientos de mezcla aplicados a la vez

Término	En una línea
LUFS	medida de sonoridad percibida. A diferencia del RMS, pondera por frecuencia y por duración
Crossfade	dos sonidos cruzándose: uno baja mientras el otro sube, sin huecos
Render	hornear un efecto en la conversión para que no cueste CPU en runtime

Antes de entregar

1. Falta `UI_Hover.wav`. Es la única fuente sin archivo. Si no llega, **borren** `SFX_UI_Hover`: es el único objeto del proyecto que hoy puede restar.
2. Generen el SoundBank (clic derecho en SoundBanks > New, arrastran los 13 Events, Generate). La consigna no lo pide, pero la clase le dedicó un apunte entero.
3. Zipeen desde el Launcher: Project options > Zip Project, tildando **todos** los tipos de archivo para que vayan los assets. Agreguen el apellido al nombre del zip.